Assignment 2

The Plaksha Orbital Pipeline Deck

CS2011: Foundations of Computer Systems

Report by- Kashika Kapoor, Jahnavi S, Vansh Jain

Introduction

This project implements a **Pipeline Hazard Simulation Applet** to visualize how instructions execute in a single-issue, in-order pipeline. The applet allows users to input instructions and observe their cycle-by-cycle movement across both **4-stage (IF, ID, EX, MEM/WB)** and **5-stage (IF, ID, EX, MEM, WB)** pipelines.

Sample of 5-stage pipeline:

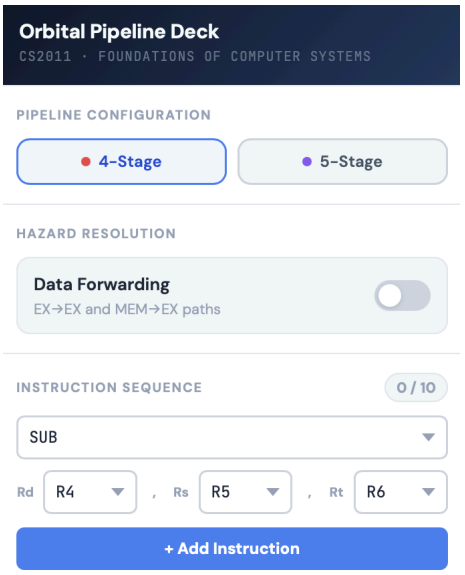
IF	ID	EX	MEM	WB

The main focus is on understanding **RAW (Read After Write) hazards**, where one instruction depends on the result of another . The simulator detects such hazards and resolves them using **stall insertion**, while also supporting **data forwarding** to reduce delays.

By clearly showing pipeline stages, stalls, and dependencies, the applet helps users easily understand how pipelining works and how hazards impact performance.

Applet Design

Interface and User Interaction



The applet is designed with a clear split between configuration and visualization to make it intuitive for users. On the left side, users can define the pipeline setup by choosing between 4-stage and 5-stage configurations and enabling or disabling data forwarding. Below this, the instruction input area allows users to construct instructions using dropdowns for opcode and registers, ensuring correct syntax and reducing input errors. The added instructions are displayed in a sequential list, making it easy to track the execution order.

This side of the interface essentially acts as the **control and input hub**, guiding users step-by-step from defining the pipeline to building the instruction sequence without overwhelming them.

Execution Flow and Visualization

The right side of the applet focuses on visually representing how instructions move through the pipeline over clock cycles. Each instruction is mapped across cycles in a table format, with stages like IF, ID, EX, MEM, and WB clearly color-coded. This makes it easy to follow instruction progression and understand overlaps in execution.

Stalls and forwarding paths are also visually indicated, helping users immediately see where delays occur and how forwarding optimizes performance. The design ensures that even

complex behaviors like overlapping instructions or delayed execution remain easy to interpret.

Performance Insights and Hazard Analysis

Alongside the pipeline visualization, the applet provides a detailed **comparison of execution with and without data forwarding**. It highlights key metrics such as total cycles, stall cycles, RAW hazards, and forwarding activity. This allows users to not only observe pipeline behavior but also analyze its efficiency.

The hazard monitoring component identifies situations like data dependencies and explains their impact on execution. By presenting both the issue (e.g., RAW hazard) and its resolution (stalling or forwarding), the applet helps users connect theoretical concepts with actual pipeline behavior.

Simulation Assumptions

Pipeline Model

- Single-issue, in-order pipeline: exactly one instruction enters the pipeline per cycle.
- Instructions proceed through stages sequentially and cannot be reordered.
- An instruction can only enter IF once the immediately preceding instruction has advanced past IF (i.e. reached ID or later).

Hazard Scope

- Only RAW (Read After Write) data hazards are simulated, as required by the assignment specification.
- WAR (Write After Read) and WAW (Write After Write) hazards are not modelled. In a single-issue in-order pipeline these hazards cannot cause actual data corruption because instructions execute in program order and register reads always complete before a later write reaches the same stage.
- Control hazards (branches, jumps) and structural hazards are outside scope. The instruction set deliberately excludes branch and jump instructions.

Register Result Availability

- Without forwarding: a register result is available only after the WB stage completes. A dependent instruction in ID must stall until the producer has finished WB.
- With EX-EX forwarding: the result computed at the end of EX is forwarded directly to the beginning of EX for the next instruction. No stall is needed for arithmetic-to-arithmetic dependencies.
- With MEM-EX forwarding: the result available at the end of MEM (for arithmetic producers) is forwarded to the beginning of EX for the consumer. This handles the case where the producer is one cycle further ahead.

Load-Use Hazard

- A LW instruction followed immediately by an instruction that reads the loaded register constitutes a load-use hazard. This hazard cannot be resolved by forwarding alone.
- Even with forwarding enabled, exactly one stall cycle is inserted because the loaded value is not available until the end of MEM, one cycle after the consumer needs it at the start of EX.

Stall Propagation

- When an instruction is stalled in ID, any instruction currently in IF is also frozen for the same cycle. This preserves in-order behaviour.
- Stall cells are labelled "STALL" in the timeline table and carry a tooltip describing the cause.

Forwarding Detection

- Forwarding is checked at the cycle when a consumer instruction attempts to advance from ID to EX.
- If the producer is in EX (EX-EX path) or in MEM (MEM-EX path) at that same cycle and the producer writes a register that the consumer reads, forwarding resolves the hazard with zero stall cycles.
- The exception is the LW load-use case described above: even with forwarding the consumer must stall one cycle, as the loaded value is only ready at the end of MEM.

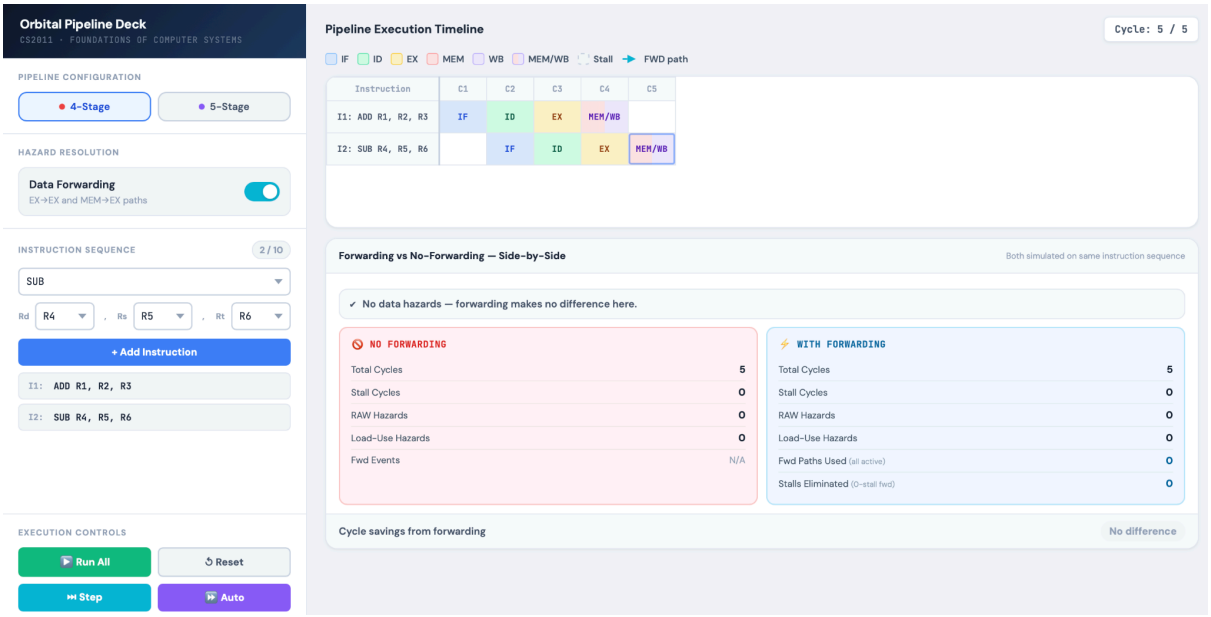
Implementation of the Test Cases

Test Case 1: No Dependency

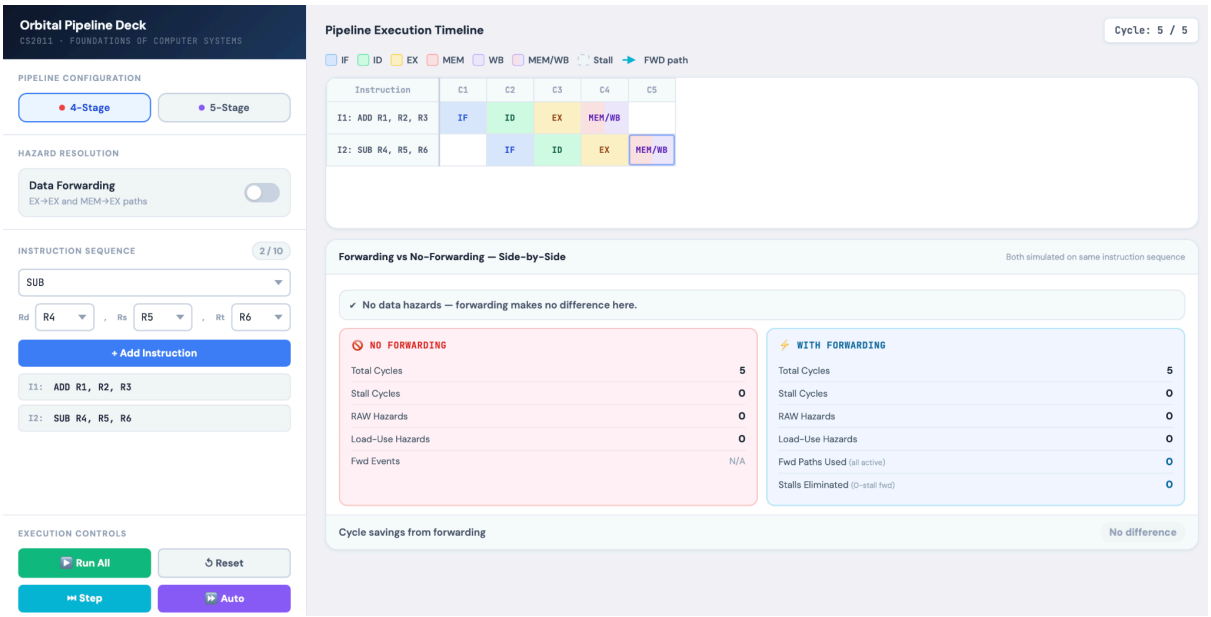
I1: add R1, R2, R3
I2: sub R4, R5, R6

- **4-Stage Pipeline Analysis**

With Data Forwarding

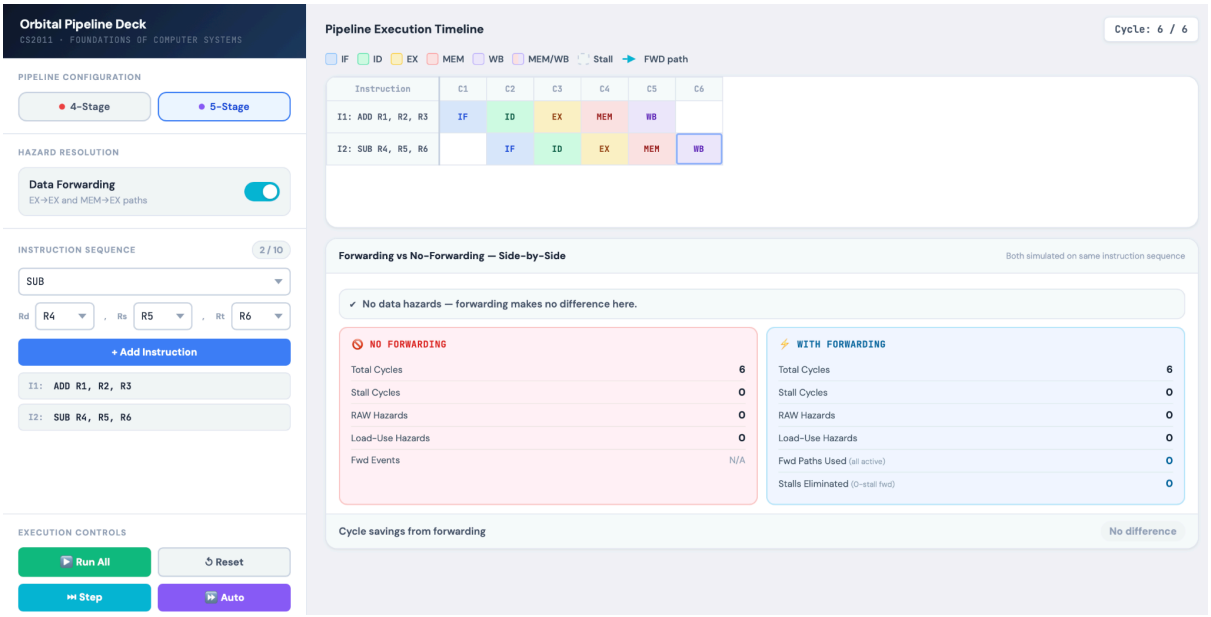


Without Data Forwarding

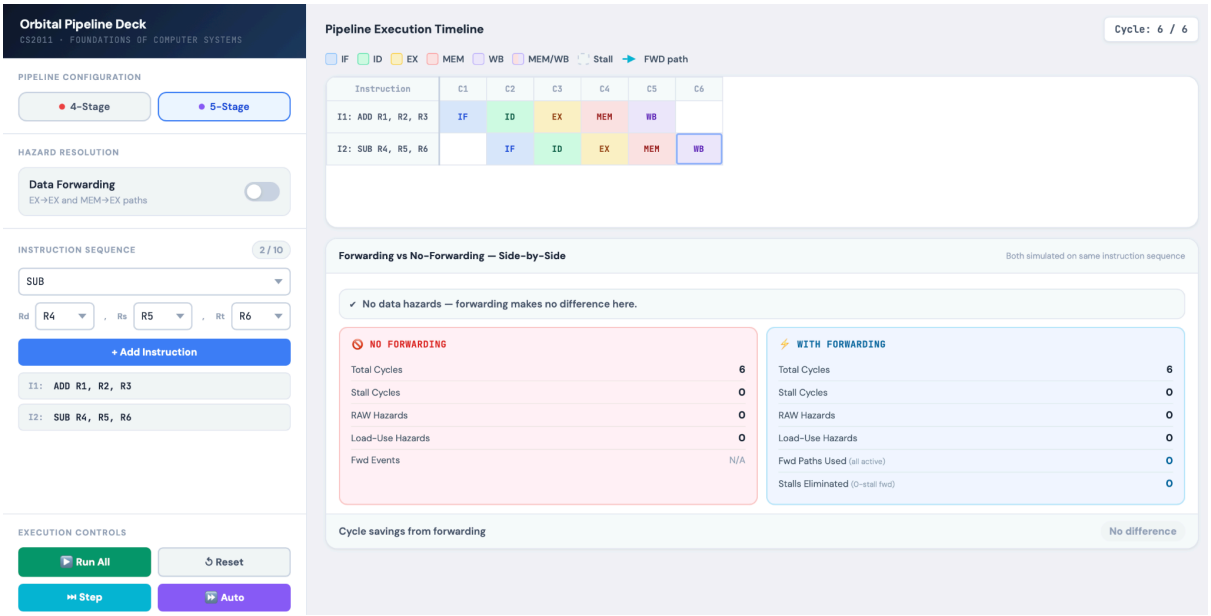


- **5-Stage Pipeline Analysis**

With Data Forwarding



Without Data Forwarding

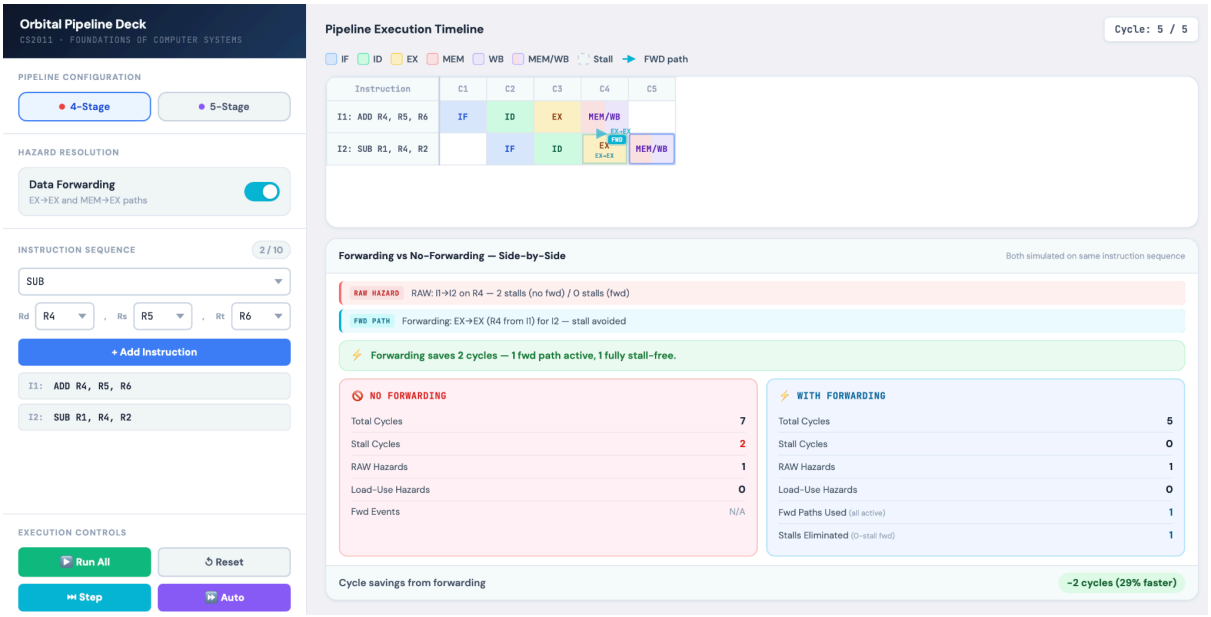


Test Case 2: RAW Hazard (ALU Dependency)

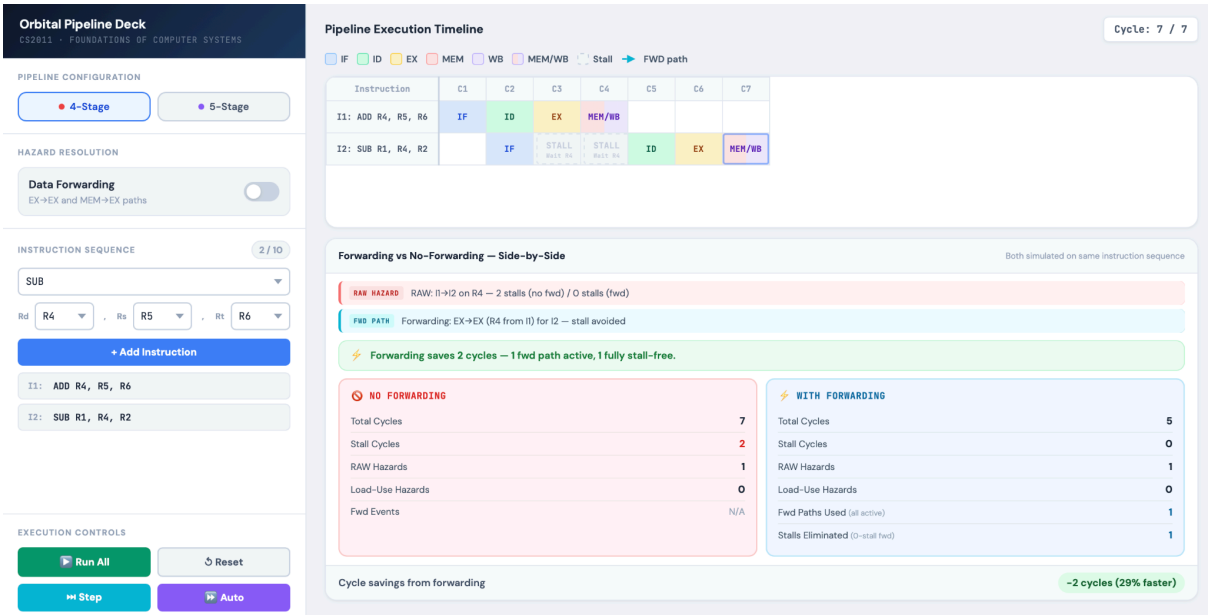
I1: add R4, R5, R6
I2: sub R1, R4, R2

- **4-Stage Pipeline Analysis**

With Data Forwarding

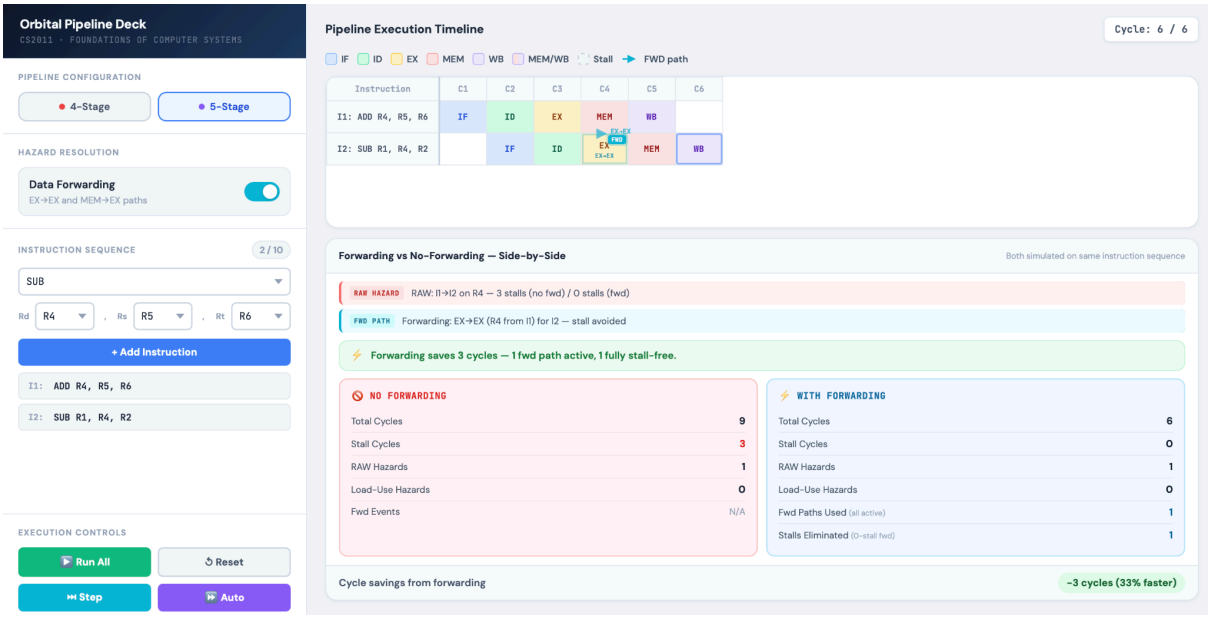


Without Data Forwarding

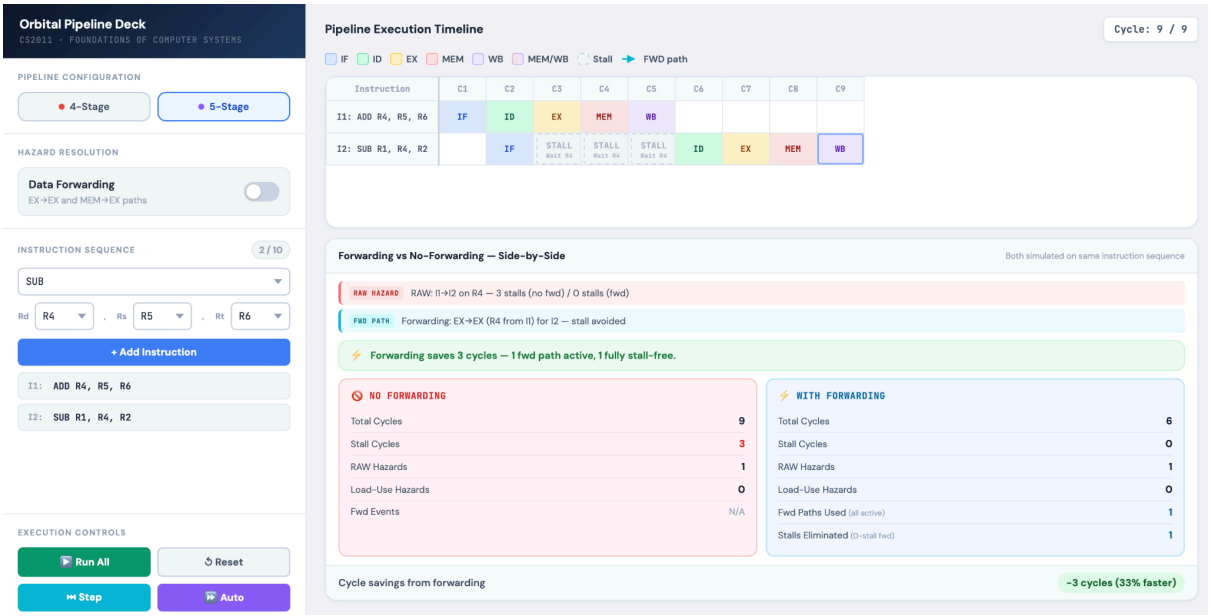


- **5-Stage Pipeline Analysis**

With Data Forwarding



Without Data Forwarding

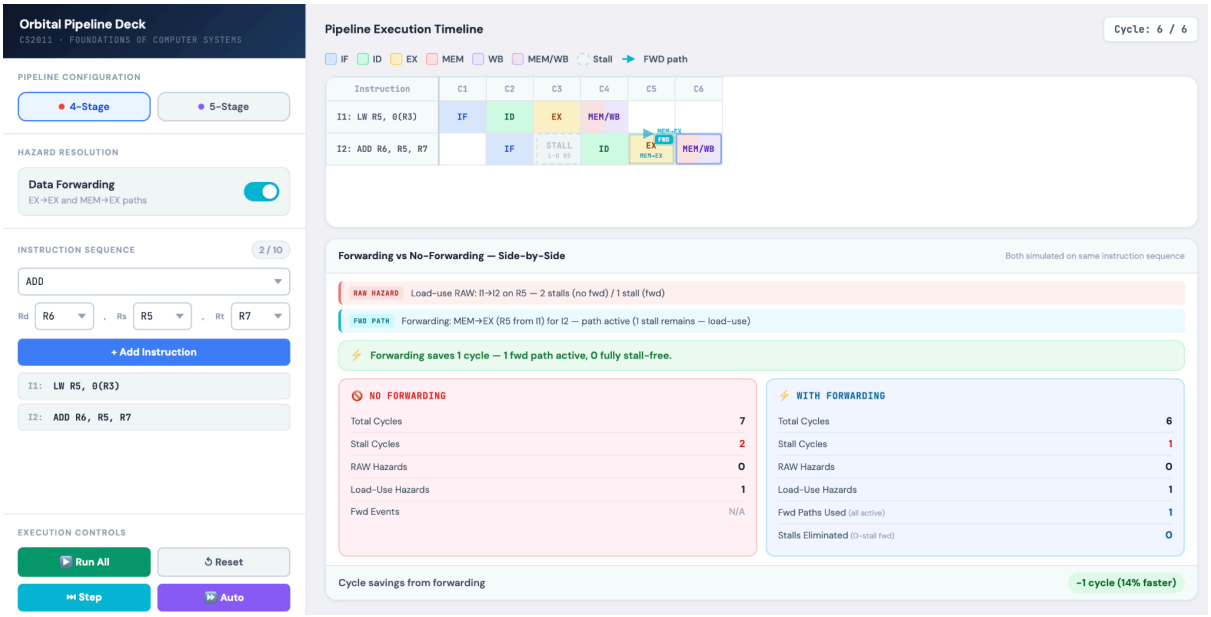


<u>Test Case 3:</u> Load-Use Hazard

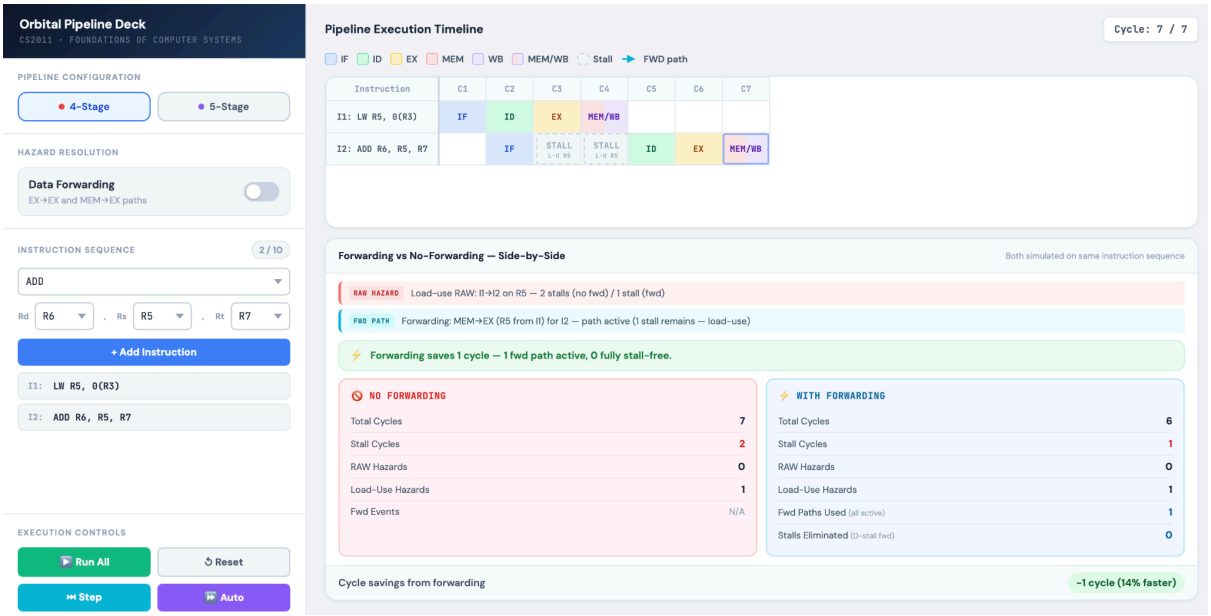
I1: LW R5, 0(R3)
I2: ADD R6, R5, R7

- **4-Stage Pipeline Analysis**

With Data Forwarding

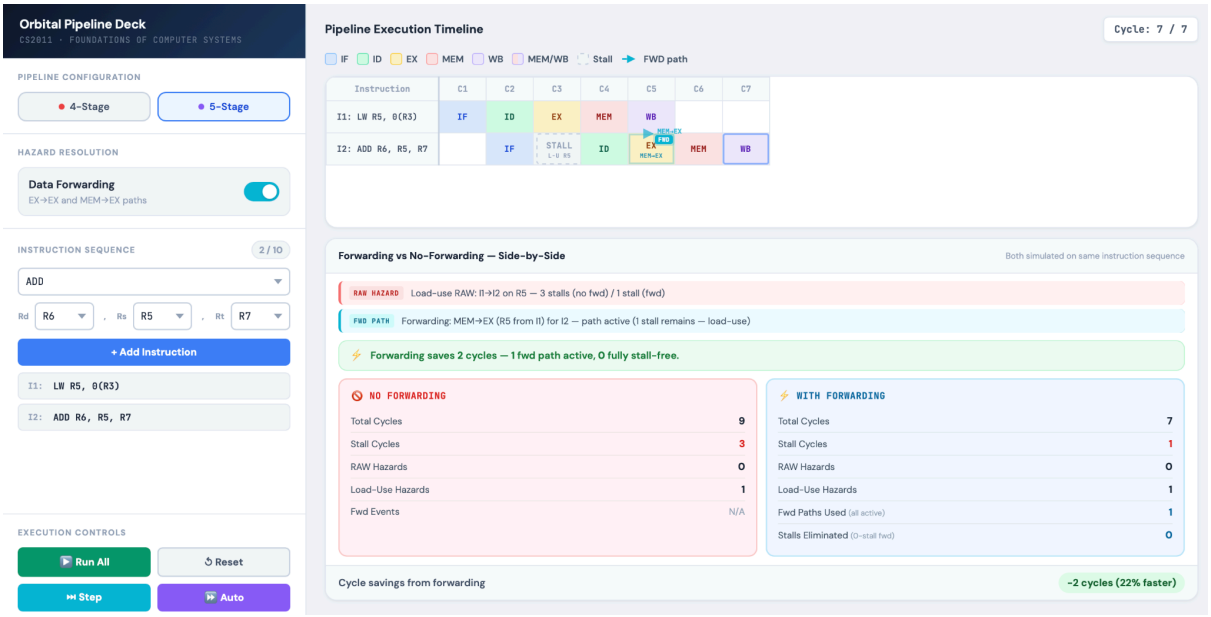


Without Data Forwarding

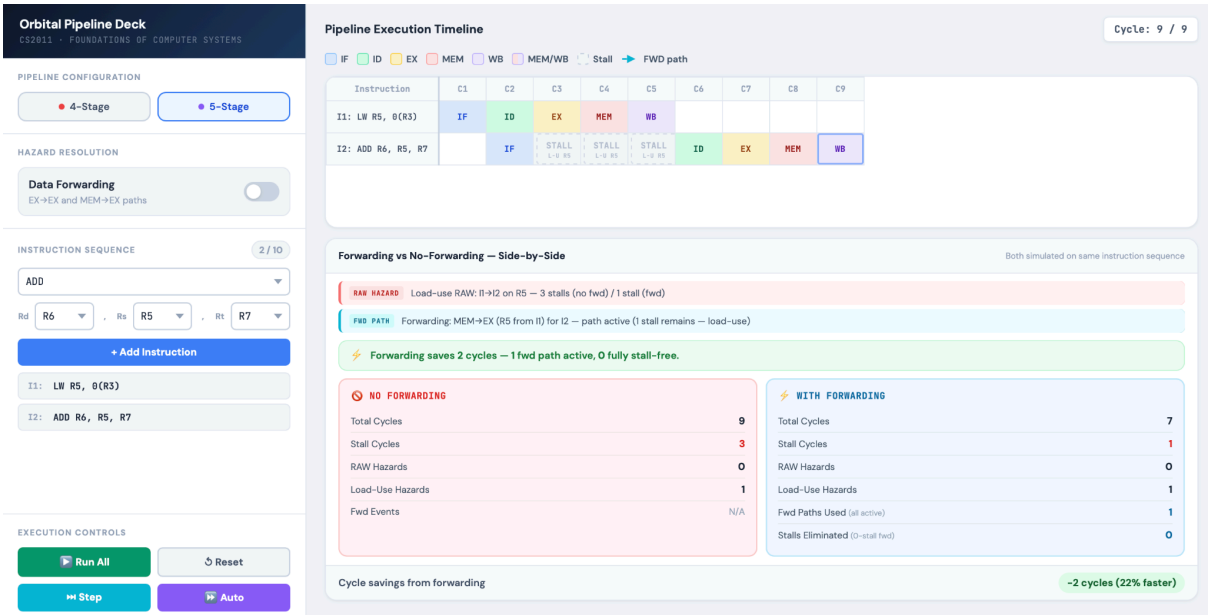


- **5-Stage Pipeline Analysis**

With Data Forwarding



Without Data Forwarding

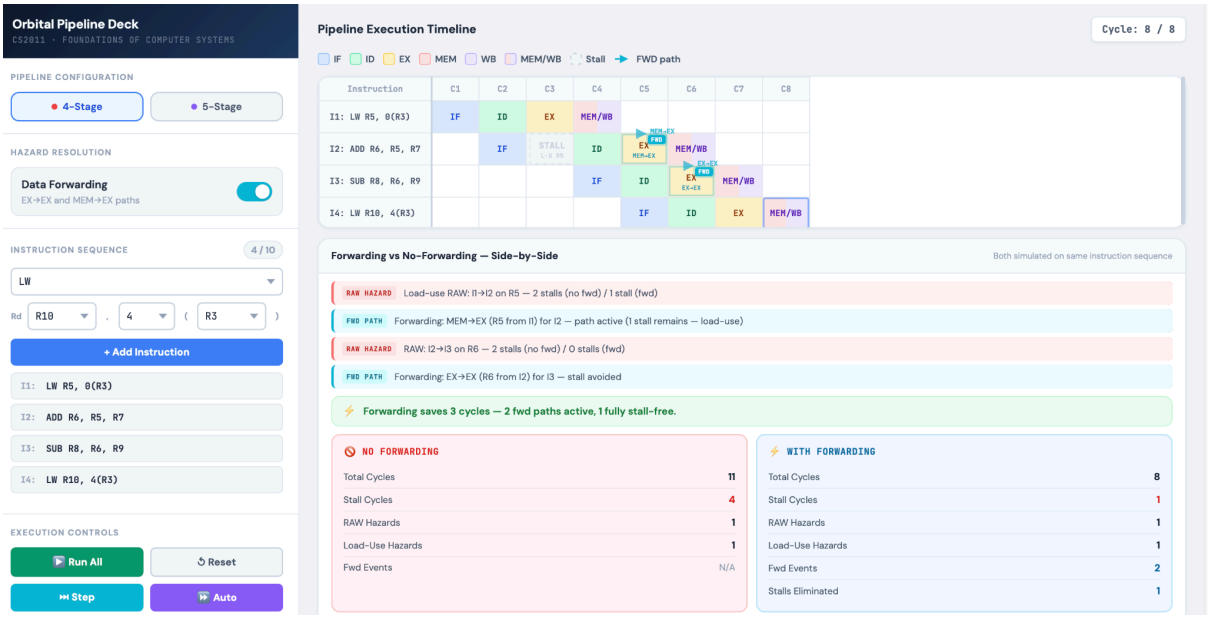


<u>Test Case 4:</u> Mixed Hazards (RAW Chain and Load-Use)

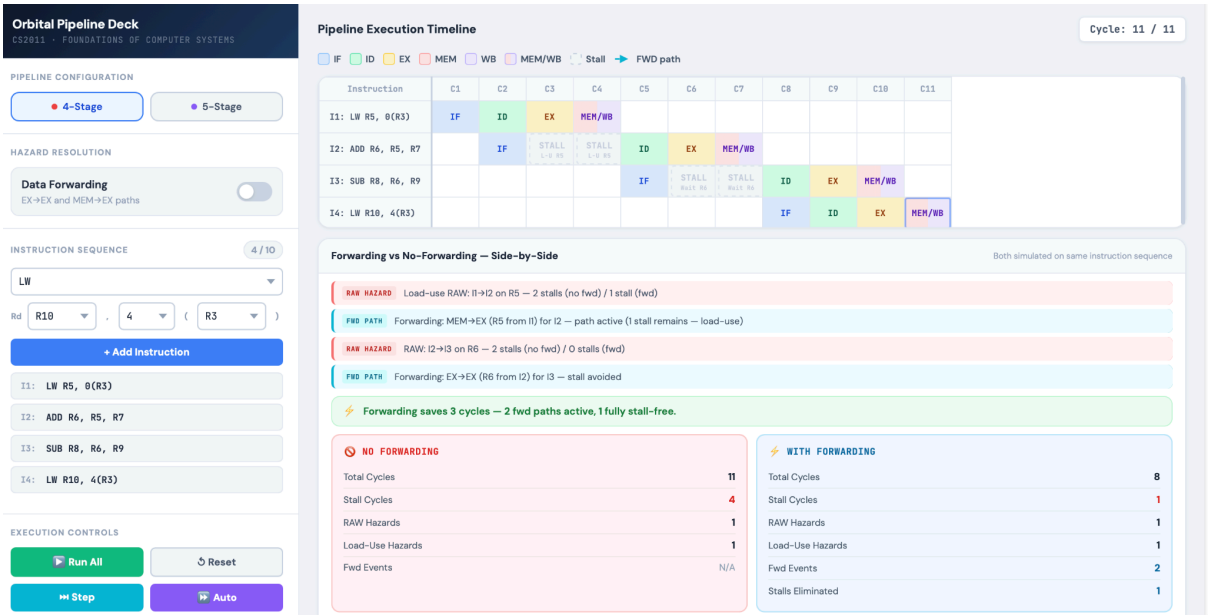
I1: LW R5, 0(R3)
I2: ADD R6, R5, R7
I3: SUB R8, R6, R9
I4: LW R10, 4(R3)

- **4-Stage Pipeline Analysis**

With Data Forwarding

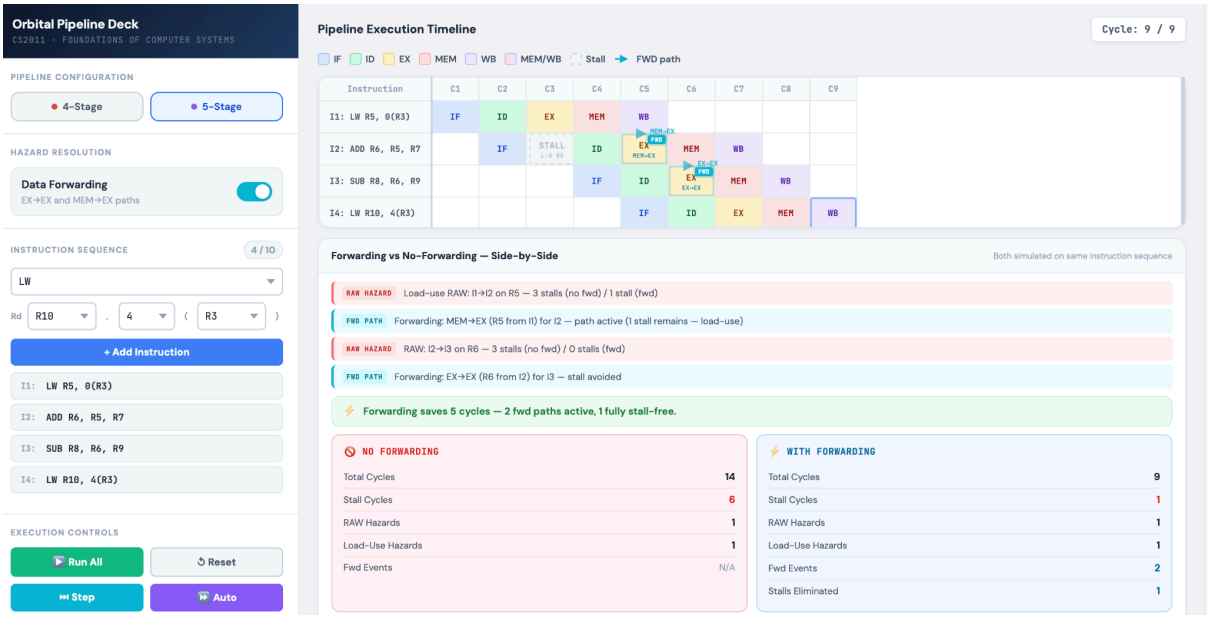


Without Data Forwarding



- **5-Stage Pipeline Analysis**

With Data Forwarding



Without Data Forwarding